
boolean



Context API

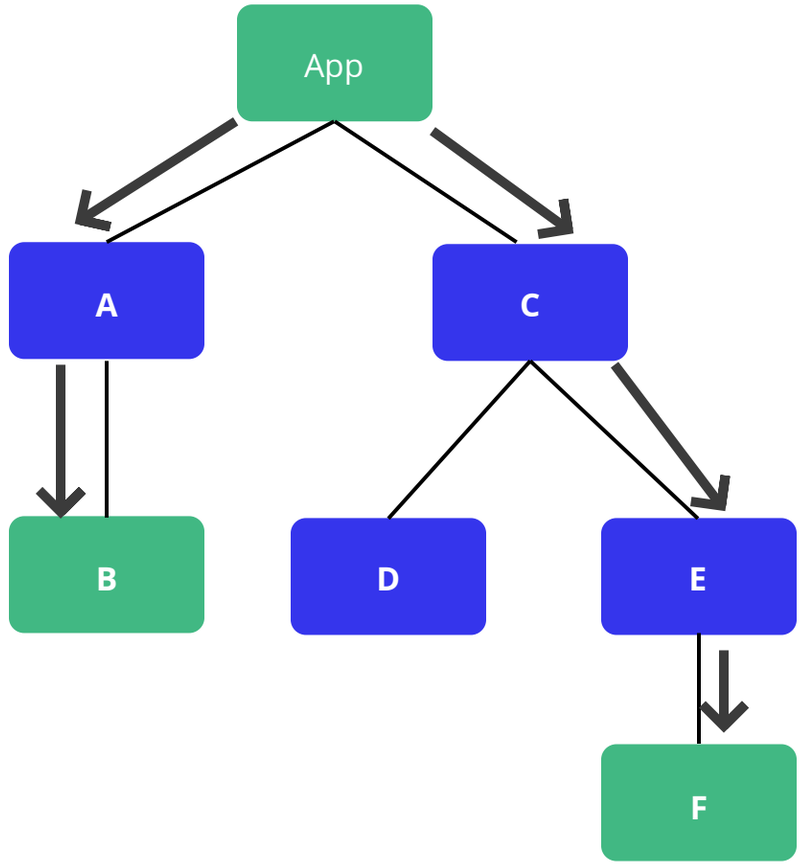
Gestire lo stato globale

Gestire lo stato globale: Context API

La gestione dello **stato globale** diventa fondamentale quando diversi componenti all'interno dell'applicazione necessitano di accedere agli stessi dati.

Supponiamo di avere diversi componenti, come **B** e **F** nel diagramma, che devono accedere a una variabile di stato chiamata "**count**".

Con le conoscenze attuali, potremmo essere tentati di dichiarare questo stato nel componente genitore comune, ad esempio "**App**", e passare questo dato attraverso **props**. Tuttavia, questo approccio potrebbe comportare il passaggio del dato attraverso componenti intermedi che non necessitano direttamente di questo stato, rendendo il codice meno chiaro e più complesso da mantenere.



⚠️ **Props Drilling**



LIVE CODING

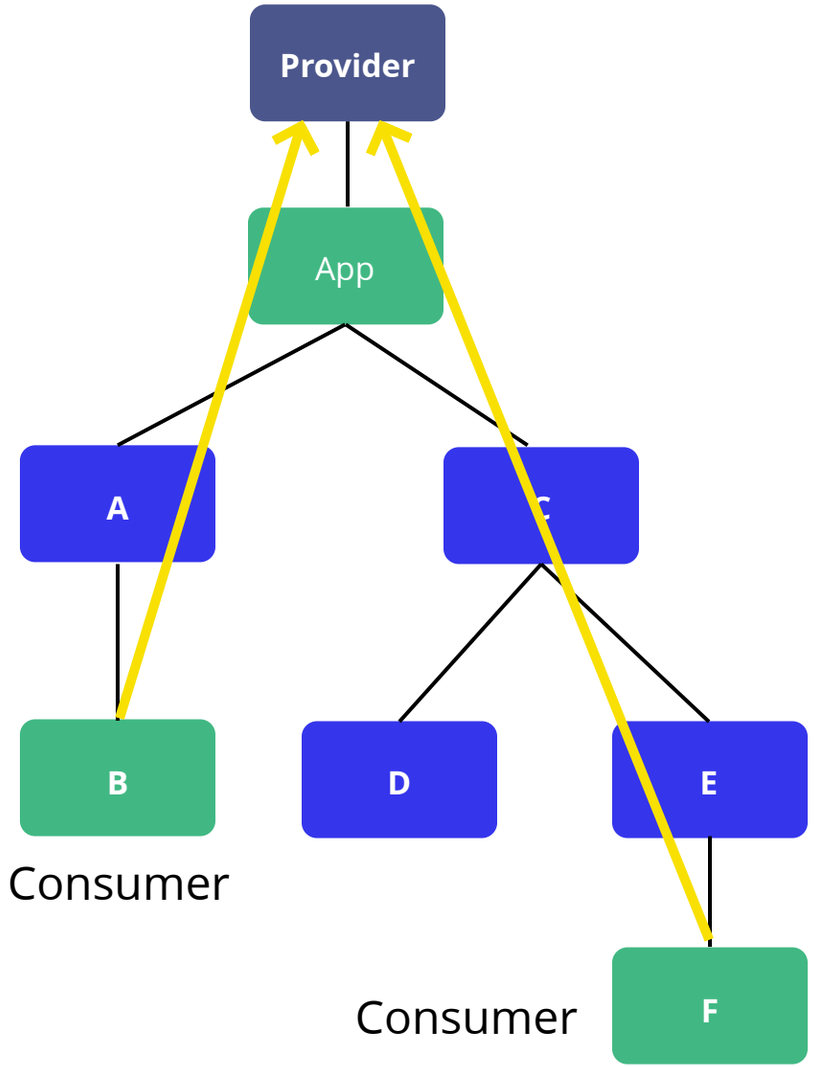
Props Drilling

Gestire lo stato globale: Context API

Grazie alla **Context API** di React, possiamo **trasmettere** dati attraverso i componenti senza la necessità di passarli manualmente tramite props.

Vediamo l'anatomia della Context API:

- **Provider**: È un componente speciale di React che consente a tutti i componenti figli di accedere al valore (**value**). Di solito, questo provider viene inserito in alto nella gerarchia dei componenti.
- **value:** Questo è il dato che vogliamo rendere disponibile a tutti i componenti figli, come **variabili di stato e funzioni**.
- **Consumers:** Sono i componenti che desiderano accedere al valore reso disponibile dal provider.



Gestire lo stato globale: Context API

1 - Definizione del contesto

All'interno di una cartella chiamata **contexts**, creiamo un file chiamato **CountContext.jsx**.

Al suo interno creiamo il nostro contesto importando da react la funzione **createContext**, che restituisce un **oggetto**.

Questo oggetto, sarà il nostro **context** e dovremo esportarlo per poterlo utilizzare altrove.

```
1 import { createContext } from "react";
2
3 const CountContext = createContext();
4
5 export default CountContext;
```

Gestire lo stato globale: Context API

2 - Trasmissione dei valori

Successivamente, importiamo l'oggetto **context** dal quale possiamo ottenere un Componente tramite la sua proprietà **Provider**.

Questo Provider sarà utilizzato per wrappare altri componenti. Tutti i componenti al suo interno (compresi i rispettivi figli) potranno accedere ai dati che definiremo nella prop `value`

```
1 import CountContext from './contexts/CountContext';
2
3 return (
4   <CountContext.Provider value={{ count: 1 }}>
5
6     // Tutti i componenti qui vedranno il value
7
8   </CountContext.Provider>
9 );
```

Gestire lo stato globale: Context API

3 - Consumare il contesto

Infine, nei componenti figli che necessitano di accedere a questi dati, detti **Consumer**, dobbiamo fare le seguenti operazioni:

1) Importiamo il **context** ed l'hook **useContext**

2) Utilizziamo l'hook **useContext** passandogli il **context** , in modo da recuperare così i dati che avevamo inserito nella prop **`value`** del provider

```
1 import { useContext } from "react";
2 import CountContext from '../../contexts/CountContext';
3
4 function MyCounter() {
5   const { count } = useContext(CountContext);
6   return <div>{count}</div>;
7 }
```



LIVE CODING

Creiamo un Context



Context API Pattern

Creiamo un custom Hook

Context API Pattern (1)

Un pattern molto comune, prevede l'inserimento di tutta la logica legata al contesto, all'interno del file del contesto stesso per semplificare al massimo la logica nei componenti che lo sfruttano!

```
1 import { createContext } from "react";
2
3 // Creaiamo il contesto
4 const CountContext = createContext();
```

Andiamo per step:

Innanzitutto, **si crea il contesto, ma non lo si esporta direttamente.**

Context API Pattern (2)

In seguito, creiamo un nostro componente **countProvider**, che sfrutta al suo interno il provider del Context.

In questo componente dovremo definire tutti i dati che vogliamo rendere disponibili a chi sarà wrappato. (ad esempio `count`)

Potremo anche rendere disponibili delle funzioni da poter richiamare (ad esempio `setCount()`.

Per renderle disponibili, dovremo inserirle nell'attributo `value` del provider

```jsx
import { createContext, useContext, useState } from "react";

const CountContext = createContext();

// Definiamo un custom Provider
function CountProvider({ children }) {
  // Aggiungiamo le varibili di
  // stato che vogliamo condividere
  const [count, setCount] = useState(0);
  return (
    <CountContext.Provider
      value={{
        count,
        setCount,
      }}
    >
      {children}
    </CountContext.Provider>
  );
}
```

Context API Pattern (3)

Similmente a quanto fatto prima, creiamo un nostro hook custom, **useCount()**, che usa al proprio interno l'hook **useContext()** di React e gli passa il nostro contesto.

Infine non ci resta che esportare il nostro customProvider ed il nostro custom hook!

Facciamo questo per evitare che i componenti interessati a usare il nostro contesto debbano sempre importare sia l'hook useContext che il nostro Context.

```jsx
import { createContext, useContext, useState } from "react";

const CountContext = createContext();

function CountProvider({ children }) {
  const [count, setCount] = useState(0);
  return (
    <CountContext.Provider
      value={{
        count,
        setCount,
      }}
    >
      {children}
    </CountContext.Provider>
  );
}


// Definiamo un hook per consumare il contesto
function useCount() {
    const context = useContext(CountContext);
    return context;
}

// Esportiamo il nostro provider ed il nostro hook
export {CountProvider, useCount}
```

Context API Pattern (4)

Ora, nel nostro file **App.jsx**, importiamo il componente **CountProvider** e lo utilizziamo per incapsulare tutti i componenti che necessitano di accedere a questo contesto.

```jsx
import { CountProvider } from "./contexts/CountContext";
import MyCounter from "./components/MyCounter";

export default function App() {
  return (
    <CountProvider>
      <MyCounter />
    </CountProvider>
  );
}
```

Successivamente, nei componenti figli, importiamo il custom hook **useCount** per recuperare i dati che vogliamo visualizzare o utilizzare.

```jsx
import { useCount } from "../contexts/CountContext";

function MyCounter() {
  const { count } = useCount();
  return <div>{count}</div>;
}
```

Context API Pattern (5)

Grazie a questo trucchetto, il codice che dovremo scrivere nei componenti che sfruttano l'hook sarà più semplice ed immediato!

```jsx
import CountContext from './contexts/CountContext';

return (
  <CountContext.Provider value={{ count: 1 }}>
    // Tutti i componenti qui vedranno il value
  </CountContext.Provider>
);
```

```jsx
import { CountProvider } from './contexts/CountContext';

return (
  <CountProvider>
    // Tutti i componenti qui vedranno il value
  </CountProvider>
);
```

```jsx
import { useContext } from "react";
import CountContext from '../../contexts/CountContext';

function MyCounter() {
  const { count } = useContext(CountContext);
  return <div>{count}</div>;
}
```

```jsx
import { useCount } from '../../contexts/CountContext';

function MyCounter() {
  const { count } = useCount();
  return <div>{count}</div>;
}
```



LIVE CODING

Refactoring del Context

ESERCIZIO